OS Security Lab:

Using John the Ripper to Crack Linux Passwords

Lab Type: Password Cracking & Security Analysis **Duration:** 60 minutes **Difficulty:** Intermediate

Introduction

In this lab, students will become familiar with the location where Linux passwords are stored and learn about tools and techniques for breaking Linux passwords.

Passwords help to secure systems running Linux and UNIX operating systems. If an attacker is able to get the root password on a Linux or UNIX system, they will be able to take complete control of that device. The protection of the root password is critical.

Key Concepts

passwd file – User accounts on a Linux system are listed in the passwd file which is stored in the `/etc` directory. The passwd file has less restrictive permissions than the shadow file because it does not store the encrypted password hashes. On most Linux systems, any account has the ability to read the contents of the passwd file.

shadow file – The shadow file also stores information about user accounts on a Linux system. The shadow file also stores the encrypted password hashes, and has more restrictive permissions than the passwd file. On most Linux systems, only the root account has the ability to read the contents of the shadow file.

auth.log – This log file tracks SSH, or Secure Shell, connections. It provides information such as IP addresses, and date and time stamps. It also tracks other events related to security, such as the creation of new user's accounts and new group accounts.

John the Ripper – John the Ripper is an extremely fast password cracker that can crack passwords through a dictionary attack or through the use of brute force.

Lab Objectives

By the end of this lab, you will be able to: - Understand the structure and purpose of `/etc/passwd` and `/etc/shadow` files - Examine file permissions on critical system files - Create user accounts and understand password storage mechanisms - Use John the Ripper to perform password cracking - Understand password salting and its security implications - Perform dictionary and brute force attacks - Analyze authentication logs for security events

Prerequisites

Required: - Linux system (Ubuntu/Debian recommended) - Root or sudo access - Basic command-line knowledge - Terminal access

Tools to be installed: - John the Ripper

Part 1: Locating and Understanding Linux Password Storage

Overview

Learn where Linux stores user account information and password hashes, and understand the security model behind password storage.

Step 1: Examining the /etc/passwd File

The passwd file contains basic user account information and is world-readable.

```
# View the contents of the passwd file
cat /etc/passwd
```

Example output:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
seed:x:1000:1000:Seed User,,,:/home/seed:/bin/bash
```

Field breakdown:

```
username:x:UID:GID:GECOS:home_directory:shell

seed           : Username
x              : Password placeholder (actual hash in /etc/shadow)
1000           : User ID (UID)
1000           : Group ID (GID)
Seed User,,,   : GECOS field (full name and comments)
/home/seed     : Home directory
/bin/bash      : Login shell
```

Step 2: Check Passwd File Permissions

View the permissions on the `/etc/passwd` file:

```
ls -l /etc/passwd
```

Output:

```
-rw-r--r-- 1 root root 2345 Oct 13 10:00 /etc/passwd
```

Understanding the permissions: - **Owner (root)**: `rw-` (read and write) - **Group (root)**: `r--` (read only) - **Others**: `r--` (read only)

Why world-readable? - Many programs need to map UIDs to usernames - Does not contain sensitive data (passwords moved to shadow file) - Commands like `ls -l` need username information

Historical note: At one time, the password was stored in the passwd file. However, due to the fact the passwd file does not have very restrictive permissions, the password is no longer stored there. Instead, there is an x present, which designates that it is stored in the shadow file.

Step 3: Examining the /etc/shadow File

The shadow file contains password hashes and is only readable by root.

```
# Try as regular user (should fail)
cat /etc/shadow
# Expected: Permission denied

# View with sudo
sudo cat /etc/shadow
```

Example output:

```
root:$6$xyz$HASH...:19000:0:99999:7:::
seed:$6$abc$HASH...:19200:0:99999:7:::
```

Field breakdown:

```
username:password_hash:last_changed:min:max:warn:inactive:expire:reserved

seed                            # 1. Username
$6$rounds=5000$salt$HASH...    # 2. Password hash
19000                           # 3. Days since 1970 when password last change
d
0                               # 4. Min days before password can be changed
99999                           # 5. Max days password is valid (99999 = no ex
piry)
7                               # 6. Days before expiry to warn user
                                # 7. Days after expiry until account disabled
                                # 8. Days since 1970 when account expires
                                # 9. Reserved field
```

Understanding password hash format:

```
$6$rounds=5000$salt$hash...

$6$              : Algorithm identifier (SHA-512)
rounds=5000      : Number of hashing iterations
```

```
salt             : Random salt data
hash...          : The actual password hash
```

Common algorithm identifiers: - `$1$` : MD5 (legacy, weak - avoid!) - `$5$` : SHA-256 - `$6$` : SHA-512 (common default) - `$y$` : Yescrypt (modern, recommended)

Security note: The shadow file has restrictive permissions (readable only by root) to protect password hashes from unauthorized access.

Part 2: Creating Test Users and Observing Changes

Step 1: Create Test User Accounts

Create users that we'll use for password cracking demonstrations.

```
# Create user 'yoda'
sudo adduser yoda
# When prompted, enter password: green
# Press Enter through the other prompts (full name, etc.)

# Create user 'chewbacca' (note: typo in original is 'chewbaca')
sudo adduser chewbacca
# When prompted, enter password: green
```

What happens during user creation: 1. New entry added to `/etc/passwd` 2. New entry added to `/etc/shadow` 3. Home directory created (`/home/yoda`, `/home/chewbacca`) 4. Default configuration files copied to home directory 5. Event logged in `/var/log/auth.log`

Step 2: Examine Changes to passwd File

```
# View the last 10 lines of passwd file
tail /etc/passwd
```

Expected output:

```
yoda:x:1001:1001:,,,:/home/yoda:/bin/bash
chewbacca:x:1002:1002:,,,:/home/chewbacca:/bin/bash
```

Key observations: - New users added to bottom of file - First new user typically gets UID 1001 - Each user gets unique UID and GID - The x indicates password stored in shadow file - Root account always has UID 0

Security implication: If another account were able to obtain a UID of 0, that account would also have root permissions.

Step 3: Examine Changes to shadow File

```
# View the last 2 lines of shadow file
sudo tail -n 2 /etc/shadow
```

Initial output (before setting passwords):

```
yoda:!:19200:0:99999:7:::
chewbacca:!:19200:0:99999:7:::
```

Understanding the ! symbol: - The ! symbol represents that the password has not been set yet - Account is locked and cannot be used for login - Password must be set before account can be used

Step 4: Set User Passwords

Set passwords for the new accounts.

```
# Set password for yoda
sudo passwd yoda
# New password: green
# Retype password: green

# Set password for chewbacca
sudo passwd chewbacca
# New password: green
# Retype password: green
```

Important notes: - For security reasons, the password will not be displayed as you type - Use simple passwords for this exercise only - **Never use simple passwords on production systems!**

Password best practices: - Minimum 8 characters (preferably 12+) - Mix of uppercase and lowercase letters - Include numbers and special characters - Avoid dictionary words - Avoid personal information

Step 5: Examine Password Hashes

```
# View the password hashes
sudo tail -n 2 /etc/shadow
```

Output:

```
yoda:$6$xyz$ABC123...:19200:0:99999:7:::
chewbacca:$6$abc$DEF456...:19200:0:99999:7:::
```

Critical observation - Password Salting: Both users have the password “green”, but the hashes are **completely different**!

Why are they different? - Each password is **salted** with random data - Salt is included in the hash (the xyz and abc parts) - Same password + different salt = different hash

Security implications:

```
Without salt:
  password123 → 482c811da5d5b4bc6d497ffa98491e38 (same for everyone)
  Attacker can use rainbow tables!
```

```
With salt:
  User1: salt_xyz + password123 → $6$xyz$ABC...
  User2: salt_abc + password123 → $6$abc$DEF...
  Different hashes! Rainbow tables are useless!
```

Why salting matters: - Prevents rainbow table attacks - Forces attackers to crack each hash individually - Makes pre-computed attack tables ineffective - Significantly increases time needed for cracking

Step 6: Monitor Authentication Logs

Check what events were logged during user creation and password changes.

```
# View recent entries in auth.log
tail /var/log/auth.log
```

Expected log entries:

```
Oct 13 10:30:15 ubuntu useradd[1234]: new user: name=yoda, UID=1001, GID=1001
Oct 13 10:30:45 ubuntu passwd[1235]: password changed for yoda
Oct 13 10:31:10 ubuntu useradd[1236]: new user: name=chewbacca, UID=1002, GID
=1002
Oct 13 10:31:30 ubuntu passwd[1237]: password changed for chewbacca
```

Note: Results may vary. The `tail` command only shows the last 10 entries. If you don't see your changes, they're still in the log, just not in the last 10 lines.

Step 7: Search Logs for Specific Events

Use grep to find specific information within log files.

```
# Search for all password changes
cat /var/log/auth.log | grep changed

# Or use grep directly
grep "password changed" /var/log/auth.log

# Search for user creation events
grep "new user" /var/log/auth.log

# Search for specific user
grep "yoda" /var/log/auth.log
```

Understanding grep: - **grep** = Global Regular Expression Print - Searches for patterns in text - Native to most Linux distributions - Extremely useful for log analysis

Part 3: Installing and Using John the Ripper

Overview

John the Ripper is an extremely powerful password cracker that can perform both dictionary and brute force attacks against password hashes.

Step 1: Install John the Ripper

```
# Install john
sudo apt-get update
sudo apt-get install john
```

What is John the Ripper? - Free, open-source password cracking tool - Available at: www.openwall.com/john/ - Comes pre-installed on security distributions like Kali Linux - Supports multiple hash formats - Can perform dictionary and brute force attacks

Step 2: Explore John's Options

```
# View available switches and options
john
```

Common John the Ripper options:

```
--wordlist=FILE      Use dictionary file
--rules              Apply word mangling rules
--incremental        Brute force mode
--format=TYPE        Specify hash format
--show               Show cracked passwords
--restore            Continue interrupted session
```

Step 3: Run John Against Password Hashes

Attempt to crack the passwords using John the Ripper.

```
# Crack passwords in /etc/shadow
sudo john /etc/shadow
```

Expected output:

```
Loaded 4 password hashes with 4 different salts (sha512crypt [64/64])
Proceeding with single crack mode
Proceeding with wordlist mode
green            (yoda)
green            (chewbacca)
green            (seed)
```

Understanding the output: - John loaded 4 password hashes - Each hash has a different salt - John tries multiple attack modes automatically: 1. **Single crack mode**: Uses username variations 2. **Wordlist mode**: Uses built-in dictionary 3. **Incremental mode**: Brute force attack

How John works: 1. Loads password hashes from shadow file 2. Tries various cracking strategies 3. For each password guess: - Applies the same hash algorithm - Includes the salt - Compares result with stored hash 4. When match found, displays the password

Step 4: View Cracked Passwords

John stores cracked passwords for later reference.

```
# View the john.pot file (password storage)
sudo cat /root/.john/john.pot
```

Example output:

```
$6$xyz$ABC123...:green
$6$abc$DEF456...:green
$6$def$GHI789...:seed_password
```

Understanding john.pot: - Stores hash:password pairs - Prevents re-cracking same hashes - Useful for later reference - Located at `/root/.john/john.pot` (when run as root)

Step 5: Attempt to Re-crack Passwords

```
# Try to crack again
sudo john /etc/shadow
```

Output:

```
No password hashes left to crack (see FAQ)
```

Why it doesn't work: - John checks john.pot file first - Already cracked hashes are skipped - Avoids wasting time on known passwords - Efficient for large password databases

Step 6: Clear John's Cache

To crack the passwords again, remove the john.pot file.

```
# Remove the john.pot file
sudo rm /root/.john/john.pot

# Now crack again
sudo john /etc/shadow
```

Output:

```
Loaded 4 password hashes with 4 different salts (sha512crypt [64/64])
green            (yoda)
green            (chewbacca)
...
```

When to clear john.pot: - Testing different cracking strategies - Educational demonstrations - Comparing crack times - Fresh start needed

Part 4: Dictionary Attacks with John the Ripper

Overview

Dictionary attacks are faster than brute force but require a good wordlist. John comes with a built-in dictionary file.

Step 1: Explore John's Built-in Dictionary

```
# View first 20 lines of password list
head -n 20 /usr/share/john/password.lst
```

Example output:

```
123456
password
12345678
qwerty
abc123
monkey
1234567
computer
...
```

About password.lst: - Contains 3,546 common passwords - Located in `/usr/share/john/` - Includes common words and patterns - Based on real-world password dumps

Step 2: Test Dictionary Attack (Fast Password)

Set a password that's in the dictionary.

```
# Change chewbacca's password to 'computer'
sudo passwd chewbacca
# New password: computer
# Retype: computer

# Run John with dictionary
sudo john /etc/shadow --wordlist=/usr/share/john/password.lst
```

Expected output:

```
Loaded 1 password hash
computer         (chewbacca)
```

Speed: Cracked in less than 1 second!

Why so fast? - "computer" is near the beginning of the dictionary - Dictionary attacks are much faster than brute force - John only needs to hash and compare each word

Step 3: View End of Dictionary

```
# View last 20 lines
tail -n 20 /usr/share/john/password.lst
```

Example output:

```
...
zaq1xsw2
1q2w3e4r
1qaz2wsx
!@#$%^&*
p@ssw0rd
```

Step 4: Test Dictionary Attack (Slow Password)

Set a password that's at the end of the dictionary.

```
# Change chewbacca's password to '1q2w3e4r'
sudo passwd chewbacca
# New password: 1q2w3e4r
# Retype: 1q2w3e4r

# Clear john.pot first
sudo rm /root/.john/john.pot

# Run John with dictionary
sudo john /etc/shadow --wordlist=/usr/share/john/password.lst
```

Monitoring progress: - Press Enter during cracking to see current word being tested - Shows password being tried and progress statistics

Expected output (after some time):

```
1q2w3e4r         (chewbacca)
```

Observations: - Takes longer than "computer" (it's at end of list) - John must try ~3,546 words before finding it - Still much faster than brute force - Progress can be monitored during cracking

Step 5: Compare Attack Methods

Brute Force vs Dictionary Attack:

Method	Speed	Coverage	Use Case
Brute Force	Very slow	100% (given enough time)	Complex, random passwords
Dictionary	Fast	Only dictionary words	Common, weak passwords

Method	Speed	Coverage	Use Case
Hybrid	Moderate	Dictionary + variations	Modified dictionary words

Cracking time examples (SHA-512): - 4-char password (brute force): Seconds - 8-char dictionary word: Seconds to minutes - 8-char complex password (brute force): Days to years - 12-char complex password (brute force): Centuries

Part 5: Advanced Topics and Best Practices

Understanding Attack Types

1. Brute Force Attack

- Tries every possible combination
- Guaranteed to find password (given enough time)
- Time increases exponentially with password length
- Most time-consuming method

2. Dictionary Attack

- Uses list of common passwords
- Much faster than brute force
- Only effective against weak passwords
- Success depends on wordlist quality

3. Rainbow Table Attack

- Uses pre-computed hash tables
- Very fast if table exists
- **Ineffective against salted passwords**
- Requires massive storage space

4. Hybrid Attack

- Combines dictionary with variations
- Adds numbers, special chars to words
- Example: “password” → “password123”, “P@ssw0rd”
- More effective than pure dictionary

Password Security Best Practices

For System Administrators:

1. **Enforce strong password policies:**

```
# Install password quality library
sudo apt-get install libpam-pwquality

# Edit PAM configuration
```

```
sudo nano /etc/security/pwquality.conf

# Set minimum length
minlen = 12

# Require character types
minclass = 4
```

2. **Implement password aging:**

```
# Set maximum password age (90 days)
sudo chage -M 90 username

# Set warning period (7 days)
sudo chage -W 7 username

# View password aging info
sudo chage -l username
```

3. **Monitor for weak passwords:**

```
# Audit passwords regularly
sudo john /etc/shadow --show

# Check for accounts with no password
sudo awk -F: '($2 == "" || $2 == "!") {print $1}' /etc/shadow
```

4. **Enable account lockout policies:**

```
# Edit PAM configuration
sudo nano /etc/pam.d/common-auth

# Add lockout policy
# auth required pam_tally2.so deny=5 unlock_time=900
```

For Users:

- ✅ Use minimum 12 characters
- ✅ Mix uppercase, lowercase, numbers, symbols
- ✅ Avoid dictionary words
- ✅ Use passphrases (e.g., “Correct-Horse-Battery-Staple”)
- ✅ Use password manager
- ✅ Enable two-factor authentication
- ❌ Don’t reuse passwords
- ❌ Don’t use personal information
- ❌ Don’t write passwords down (unless in secure vault)

Security Implications

Why password cracking matters:

1. **Penetration Testing:**
    - Test organization's password policies
    - Identify weak passwords before attackers do
    - Demonstrate need for security training
2. **Forensics:**
    - Recover lost passwords (authorized)
    - Access encrypted evidence
    - Investigate security incidents
3. **Security Awareness:**
    - Educate users about password strength
    - Demonstrate attack techniques
    - Motivate policy compliance

Ethical considerations: - Only test systems you own or have permission to test - Password cracking without authorization is illegal - Use knowledge for defensive purposes only - Respect privacy and confidentiality

John the Ripper Advanced Features

Custom wordlists:

```
# Create custom wordlist
cat > mywords.txt << EOF
company2024
Welcome123
Admin@2024
EOF

# Use custom wordlist
sudo john /etc/shadow --wordlist=mywords.txt
```

Rule-based attacks:

```
# Use mangling rules
sudo john /etc/shadow --wordlist=password.lst --rules

# Rules apply variations like:
# - Capitalization: password → Password
# - Appending: password → password123
# - Substitution: password → p@ssword
```

Format-specific cracking:

```
# Specify hash format
sudo john /etc/shadow --format=sha512crypt

# List supported formats
john --list=formats
```

Session management:

```
# Save session
sudo john /etc/shadow --session=mysession

# Restore interrupted session
sudo john --restore=mysession
```

Lab Summary and Review

Key Concepts Learned

Password Storage: - User accounts listed in `/etc/passwd` (world-readable) - Password hashes stored in `/etc/shadow` (root-only) - Hashes are salted to prevent rainbow table attacks - Different salt = different hash (even for same password)

Password Cracking: - John the Ripper performs dictionary and brute force attacks - Dictionary attacks are faster but limited to wordlist - Brute force is slower but comprehensive - Cracked passwords stored in `john.pot` file

Security Monitoring: - Authentication events logged in `/var/log/auth.log` - Password changes create log entries - Regular log monitoring essential for security

Best Practices: - Use strong, complex passwords (12+ characters) - Avoid dictionary words - Implement password policies - Regular password audits - Monitor authentication logs

Assessment Checklist

By completing this lab, you should be able to:

- ☐ Explain the difference between `/etc/passwd` and `/etc/shadow`
- ☐ Understand why password hashes are salted
- ☐ Create user accounts on Linux systems
- ☐ View and interpret password hashes
- ☐ Install and use John the Ripper
- ☐ Perform dictionary attacks on password hashes
- ☐ Understand the limitations of rainbow table attacks
- ☐ Monitor authentication logs for security events
- ☐ Explain password cracking techniques
- ☐ Implement strong password policies

Lab Assignment: Hydra Password Cracking Tool

Task: Play around with **Hydra Password Cracking Tool**, try various attacking scenarios and write a report about use cases.

Suggested Services to Test: 1. SSH (Secure Shell) 2. FTP (File Transfer Protocol) 3. HTTP/HTTPS (Web authentication) 4. RDP (Remote Desktop Protocol) 5. MySQL/PostgreSQL databases 6. SMTP (Email services) 7. VNC (Virtual Network Computing)

Important Notes: - Only test systems you own or have explicit permission to test - Use isolated lab environments (VMs, containers) - Never attack production systems - Document all activities for legal protection

Additional Resources

Tools for Password Security

Password Crackers: - John the Ripper - Offline hash cracking - Hashcat - GPU-accelerated cracking - Hydra - Online service attacks - Medusa - Parallel login brute-forcer

Password Generators: - pwgen - Generate random passwords - apg - Automated password generator - KeePass - Password manager with generator

Security Testing: - CrackStation - Online hash lookup - Have I Been Pwned - Check if password was compromised

Further Reading

- NIST Password Guidelines
- OWASP Authentication Cheat Sheet
- Linux Password Security Best Practices
- Rainbow Table Attacks Explained

Practice Resources

- TryHackMe - Password Cracking Rooms
- HackTheBox - Password Cracking Challenges
- OverTheWire - Wargames with password challenges

Troubleshooting

Common Issues

Issue: “Permission denied” when viewing `/etc/shadow`

```
# Solution: Use sudo
sudo cat /etc/shadow
```

Issue: John doesn't crack passwords

```
# Check if already cracked
sudo john /etc/shadow --show

# Try with wordlist
sudo john /etc/shadow --wordlist=/usr/share/john/password.lst

# Try incremental mode
sudo john /etc/shadow --incremental
```

Issue: Can't find john.pot file

```
# Check default location
sudo ls -la /root/.john/

# Or search for it
sudo find / -name "john.pot" 2>/dev/null
```

Issue: John the Ripper not installed

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install john

# Or download from official site
wget https://www.openwall.com/john/k/john-1.9.0-jumbo-1.tar.gz
```

Conclusion

In this lab, you learned:

1. **Password Storage:** Linux stores user account names in `/etc/passwd` and password hashes in `/etc/shadow`
2. **Password Salting:** Each password hash includes a random salt, making rainbow table attacks ineffective
3. **Attack Types:**
    - Brute force attacks try all possibilities
    - Dictionary attacks use wordlists
    - Rainbow tables don't work against salted hashes
4. **John the Ripper:** Powerful password cracking tool that can perform both dictionary and brute force attacks
5. **Security Monitoring:** All account changes are recorded in `/var/log/auth.log`

6. **Best Practices:** Strong passwords, proper policies, and regular auditing are essential for system security

Remember: Password cracking tools are powerful but must only be used ethically and legally. Always obtain proper authorization before testing any systems.

End of Lab